

[← Go Back](#)

Hardware

GIGA R1 WiFi



Tutorials

[Arduino GIGA R1 User Manual](#)[Guide to GIGA R1 Advanced ADC/DAC and Audio Features](#)[GIGA R1 Camera Guide](#)[Dual Core Debugging the GIGA R1 WiFi](#)[Guide to GIGA R1 Dual Cores](#)[Getting Started with GIGA R1 WiFi](#)[MicroPython on the GIGA R1](#)[Guide to Arduino GIGA USB Features](#)[GIGA R1 WiFi Network Examples](#)[Home](#) / [Hardware](#) / [GIGA R1 WiFi](#) / [Guide to GIGA R1 Dual Cores](#)

Guide to GIGA R1 Dual Cores

Learn how to access and control the M4 and M7 cores on the GIGA R1, and how to communicate between them using RPC.

Author: Karl Söderby, Ibrahim Abdalkader
Last revision: 12/04/2024

The GIGA R1's [STM32H747XI](#) has two cores, the M4 and the M7. Each core can be programmed individually, with M7 acting as the main processor, and the M4 as a co-processor.

The M7 is referred to as the main processor due its superior hardware features, as well as it is required to run to boot the M4 core (you boot the M4 from within the M7).

These two cores can run applications in parallel, for example, running a servo motor on one core, and a display on another, without blocking each other. In a single core, such operations would slow down the program, resulting in lesser performance.

The M4 and M7 cores are programmed with separate sketches, using the same serial port. In the Arduino IDE, you can select the core you want to program, and then upload the sketch you want to run on that specific core.

ON THIS PAGE

Goals

[Hardware & Software Needed](#)[M4 Support](#) +[Pin Priority](#)[Programming M4/M7](#) +[Identifying the Current CPU Core](#)[Remote Procedure Call \(RPC\)](#) +[RPC Examples](#) +[RPC Library API](#) +[RPC Serial API](#) +[Help](#)

In this guide you will discover:

How to configure and program the M4/M7 cores and conventional approaches to do so.

How to boot the M4 core.

How to communicate between the cores via Remote Procedure Call (RPC).

Using the RPC Library with MicroPython.

Useful examples based on the dual core & RPC features.

The `RPC` library API.

Hardware & Software Needed

GIGA R1 WiFi

Arduino IDE

Arduino GIGA Board Package installed.*



*For instructions on how to install the GIGA Board Package, follow the [Getting Started with GIGA R1 guide](#).

M4 Support

The M4 processor can access most of the peripherals that the M7 can access, with some exceptions.

The M4 supports:

I2C

SPI

UART

CAN

DAC

ADC

The M4 does **not** support:

Wi-Fi®

[Arduino Cloud](#) sketches

Serial over USB*

*Serial communication over USB on the M4 can be enabled only by setting up an RPC that allows the M4 & M7 cores to



communicate. Using

`RPC.print()` (M4) and

`RPC.read()` (M7) helps

achieve this. See [RPC Serial Example](#).

Boot / Disable M4

The M4 core is by manufacturing default, disabled when booting the board. The M4 core can however be booted by using the `RPC.begin()` command, which includes the necessary functions to boot the M4 core. See the [RPC.cpp source file](#) for more details.

Boot / Disable M7

The M7 is booted by default and there is currently **no option to disable** this core.

Peripheral Interference

When booting the M4, the M4 will execute the sketch that has been uploaded to its flash memory. It is a good idea to track what type of code you are running on the M4, as you may create interference between different peripherals. If you run simply a blank sketch on the M4, it should **not** create any interference.

An example of this is if you use the `CAN` library. If you are running a CAN application on the **M7**, you will disrupt it if you enable it on the **M4**. The dual core feature is not intended for using the same peripheral, bus etc.

Tip: name your sketches with a `_M4` and `_M7` suffix/prefix,



and create an initialization sequence. E.g. blink the blue LED three times whenever the M4 boots up.

Pin Priority

As the M7 and M4 core share their pins, which one gets priority to the pin? It can be assumed that as the M7 is the more powerful core, it gets first access.

This is however **not true** as pin priority is random. If both cores tries to access the same pin (e.g. `D27`), it is simply random who gets access.

When developing dual core



applications, it is a good idea avoiding using the same pins & peripheral for many reasons.

Programming M4/M7

When programming the GIGA R1 WiFi's M7 and M4, we **create a sketch for each core**, and program them like we would program two individual Arduino boards. As only a single serial port is available, we need to specify which core we want to target.

Some essential things to consider when programming the cores are:

You need to **partition the memory**, allocating flash memory to the M4 core.

You need to select the **target core**, which is either **Main Core** or **M4 Co-processor**.

The M4 does not *directly* support serial over USB. To enable it, we need to use RPC (see [RPC Serial example](#)).

When writing multiple sketches, there are some things to consider to make your development experience easier:

Name your sketches with either `_M4` or `_M7` suffix or prefix. This will make it easier if the code is intended to be shared with others.

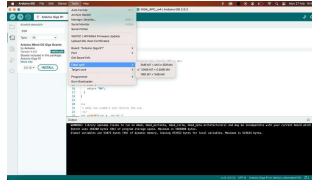
Consider having a starting sequence (e.g. the blue LED blinking 3 times), whenever a core is initialized.

Always include `RPC.begin()` on your M7 core sketch.

Partitioning The Flash Memory

To allocate the flash memory for the M4, the flash memory can be partitioned. This is done by navigating to **Tools > Flash Split** in the IDE.

Note that the flash memory is the space where the application  code (your sketch) is stored. It is not the RAM memory (which is significantly lower).



Flash partitioning in the IDE.

2MB M7 + M4 in SDRAM

(default) - this option is the default configuration, which is for programming the M7 only. This allocates no flash memory to the M4.

1.5MB M7 + 0.5MB M4 - useful when larger amount of flash memory is required on the M7.

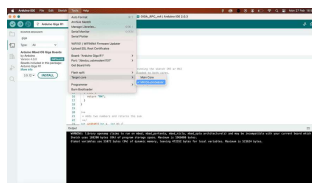
1MB M7 + 1MB M4 - useful when you need to balance the flash memory equally between the M4 and M7 cores.



It is required to use option 2 or 3 if you intend to program the M4 via the IDE, as the default option provides no memory allocation for the M4.

Target Core

To select the core you want to program, navigate to **Tools > Target Core** in the IDE.



Flash partitioning in the IDE.

Here you can choose between:

Main Core - this is the M7 core, the main processor on the board

the board.

Uploading

As both cores share the same serial port, choosing the **Flash Split + Target Core** is required so that the program is uploaded to the correct core.

Uploading is no different than to any other Arduino board: simply click the upload button and wait for it to finish.

Booting M4 Core

The M4 core does not boot by itself as it requires interaction from the M7 core. This boot function is built into the `RPC` library, and needs to be included in the sketch uploaded to the M7:

```
1 #include <RPC.h>
2
3 void setup() {
4     RPC.begin(); //boots M4
5 }
6 void loop(){
7 }
```

Once the M4 is booted from the M7, both cores will run in parallel, much like two Arduinos sharing the same board.

Writing Over Existing Sketch

Uploading new sketches works the same as a typical upload procedure.

current sketch running on the core you upload to.

Identifying the Current CPU Core

The `RPC.cpu_id()` method can be used to identify current CPU core running the sketch. This can be used to run different code paths based on the CPU core ID. For example, the following sketch blinks the blue LED if the current core is the Cortex-M7, and the Green LED if the current core is the Cortex-M4:

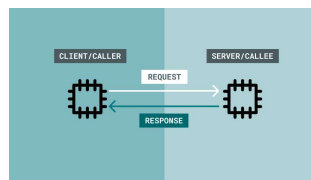
```
1  /*
2  GIGA R1 WiFi - Core ide
3
4  This simple sketch blin
5  You will need to upload
6
7  It checks whether curre
8  the blue LED or the gre
9
10 As the M4 is booted whe
11 the M4 sketch will run
12 finishes on the M7.
13 */
14
15 #include <RPC.h>
16
17 void setup() {
18   pinMode(LEDDB, OUTPUT)
19   pinMode(LEDG, OUTPUT)
20
21   if (RPC.cpu_id() == C
22       blink(LEDDB, 100); /
23   } else {
24       blink(LEDG, 100); /
25   }
26 }
27
28 void loop() {
```


Remote Procedure Call (RPC)

RPC is a method that allows programs to make requests to programs located elsewhere. It is based on the client-server model (also referred to as caller/callee), where the client makes a request to the server.

An RPC is a synchronous operation, and while a request is being made from the caller to another system, the operation is suspended. On return of the results, the operation is resumed.

The server side then performs the subroutine on request, and suspends any other operation as well. After it sends the result to the client, it resumes its operation, while waiting for another request.



Request routine.

RPCs in the Arduino Environment

At the moment, only a limited amount of boards supports RPC, as in this context, it is designed to be a communication line between **two cores**. The GIGA R1 is one of them.

What makes this implementation possible is the `RPC` library ([see API section](#)), which utilises the `rpclib` C++

The library makes it possible to set up either of the M4/M7 cores as a server/client, where remote calls can be made between them. This is done by "binding" a function to a name on the server side, and calling that function from the client side.

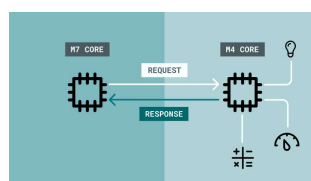
On the server side, it could look like this:

```
1 //server side, for example
2 int addFunction(int a, int b)
3 {
4   return a + b;
5 }
6 RPC.bind("addFunction", addFunction);
```

On the client side, it could look like this:

```
1 int x = 10;
2 int y = 10;
3
4 RPC.call("addFunction", x, y);
```

When `call()` is used, a request is sent, it is processed on the server side, and returned. The `x` and `y` variables are used as arguments, and the result returned should be 20 (10+10).



Communication between
M7 and M4 core.

Using the RPC Library with MicroPython

The `msgpackrpc` library provides the same functionality as the Arduino RPC library for MicroPython, i.e., it allows the binding of local functions or objects, starting the M4 core, and invoking remote calls from Python scripts. This library and its supporting features are enabled by default on all compatible Arduino boards, starting with **MicroPython release v1.23**, and require no external dependencies to use. However, there are a few restrictions to using the RPC library with MicroPython:

- Arduino sketches can only run on the M4 core.

- SDRAM-based firmware is **not** supported^[1].

- Flash-based firmware must use a **1.5MB M7 + 0.5MB M4** flash partitioning scheme.

The following sections introduce the `msgpackrpc` library API and some use cases in detail.

1. Although the `msgpackrpc` library does support loading firmware images to SDRAM, the firmware currently generated for the M4 core with an SDRAM target does not work. This issue may be fixed in future releases.

The `msgpackrpc` Library

The `msgpackrpc` library is the RPC library's counterpart for MicroPython, and it provides the same functionality as the Arduino RPC library. The first steps to using the `msgpackrpc` library, are

```
1 import msgpackrpc
2
3 # Create a MsgPackRPC object
4 rpc = msgpackrpc.MsgPackRPC()
```

The RPC object created above can then be used to bind Python callables, start the M4 core and invoke remote calls from MicroPython scripts.

Binding Python Functions, Callables and Objects

Any Python callable object (such as functions, bound methods, callable instances, etc.) can be invoked by the remote core. To allow the remote core to invoke a callable, it must be first bound to a name. The following example binds a function to the name `sub`:

```
1 def sub(a, b):
2     return a - b
3
4 # Register a function to the remote core
5 rpc.bind("sub", sub)
```

Note that the name a callable is bound to does not have to match its name.

Similarly, an object's bound method can also be bound to a name. For example:

```
1 foo = Foo()
2 rpc.bind("sub", foo.add)
```

Both of those functions can be called in the same way from the Arduino sketch:

```
1 int res = RPC.call("sub",
```

Objects can also be bound to allow their methods to be called by the remote core. When an object is passed to `bind()`, all of its public methods (the ones that don't start with an `_`) are bound to their respective qualified names. For example, the following code binds the methods of an object of class `Foo`:

```
1 class Foo:
2     def __init__(self):
3         pass
4
5     def add(self, a, b):
6         return a + b
7
8     def sub(self, a, b):
9         return a - b
10
11 # Register an object of Foo
12 rpc.bind("foo1", Foo())
```

Now the object's methods can be invoked by the Arduino sketch using their fully qualified name, for example:

```
1 int res1 = RPC.call("foo1")
2 int res2 = RPC.call("foo1")
```

Starting the M4 Core From Micropython

The next step is starting the M4 core by calling `MsgPackRPC.start()` with the firmware entry point as an argument:

```
1 # Start the remote process
2 rpc.start(firmware=0x08180000)
```

This function will start the remote core (the M4), boot it from the specified firmware address, and wait for the core to be ready to communicate before it returns. The default arguments passed to this function are compatible with the Arduino RPC library and do not need to be changed for the purposes of this tutorial. Note that the firmware address used is a flash address, where the M4 firmware starts, and it's the same one used for the flash split of `1.5MB M7 + 0.5MB M4`.

Calling Remote Functions From Micropython

Once the M4 core is started, the `MsgPackRPC` object can be used to invoke its remote functions. Remote calls can be invoked synchronously, i.e., the call does not return until a response is received from the other side, or asynchronously. In this case, the call returns a `Future` object

that must be joined at some point to read back the call's return value.

```
1 # Perform a synchronous call
2 res = rpc.call("add", 1, 2)
3
4 # Perform an asynchronous call
5 f1 = rpc.call_async("add", 1, 2)
6
7 # The Future object returns the result
8 print(f1.join())
```

That covered most of the `msgpackrpc` library's API and use cases. For a complete example, see the [MicroPython RPC LED Example](#). For more examples and applications, see the `msgpackrpc` [repository](#).

RPC Examples

In this section, you will find a series of examples that is based on the `RPC` library.

USB Serial Using RPC

The `Serial.print()` command only works on the **M7 core**. In order to print values on the **M4**, we need to:

Use `RPC.println()` on the M4. This will print the values to the RPC1 stream.

Use `RPC.available()` and `RPC.read()`.

M4 Sketch:

```
1 #include <RPC.h>
2
3 void setup() {
4   RPC.begin();
5 }
6
7 void loop() {
8   RPC.println("Printed fr
9   delay(1000);
10 }
```

M7 Sketch:

```
1 #include <RPC.h>
2
3 void setup() {
4   Serial.begin(9600);
5   RPC.begin();
6 }
7
8 void loop() {
9   String buffer = "";
10  while (RPC.available())
11    buffer += (char)RPC.r
12  }
13  if (buffer.length() > 0
14    Serial.print(buffer);
15  }
16 }
```

RPC Sensor

This example demonstrates how to request a sensor reading from one core to the other, using:

M4 as a client.

M7 as a server.

M4 Sketch:


```
1 #include "Arduino.h"
2 #include "RPC.h"
3
4 using namespace rtos;
5
6 Thread sensorThread;
7
8 void setup() {
9     RPC.begin();
10    Serial.begin(115200);
11
12    /*
13     Starts a new thread t
14    */
15    sensorThread.start(re
16 }
17
18 void loop() {
19
20 }
21
22 /*
23 This thread calls the s
24 every second. Result is
25 */
26 void requestReading() {
27     while (true) {
28         delay(1000);
29     }
```

M7 Sketch:

```
1 #include "Arduino.h"
2 #include "RPC.h"
3
4 void setup() {
5     RPC.begin();
6     Serial.begin(115200);
7
8     //Bind the sensorRead
9     RPC.bind("sensorRead"
10 }
11
12 void loop() {
13     // On M7, let's print
14     // Buffer it, otherwi
15     String buffer = "";
16     while (RPC.available(
17         buffer += (char)RPC
18     }
19     if (buffer.length() >
20         Serial.print(buffer
21     }
22 }
23
24 /*
25 Function on the M7 that
26 */
27 int sensorRead() {
28     int result = analogRe
```

RPC Servo Motor

This example demonstrates how to request a servo motor on another core to move to a specific angle, using:

M4 as a client.

M7 as a server.

Each example is written as a **single sketch** intended to be uploaded to **both cores**.

M4 sketch:

```
1 #include "Arduino.h"
2 #include "RPC.h"
3
4 using namespace rtos;
5
6 Thread servoThread;
7
8 void setup() {
9     RPC.begin();
10    Serial.begin(115200);
11
12    /*
13     Starts a new thread t
14    */
15    servoThread.start(req
16 }
17
18 void loop() {
19 }
20
21 /*
22 This thread calls the s
23 every second, passing t
24 */
25 void requestServoMove()
26 while (true) {
27     //Read a pot meter
28     int rawAnalog = ana
```

M7 sketch:

```
1 #include "Arduino.h"
2 #include "RPC.h"
3 #include <Servo.h>
4
5 Servo myservo;
6
7 void setup() {
8     RPC.begin();
9     myservo.attach(5); //
10
11     Serial.begin(115200);
12
13     //Bind the servoMove(
14     RPC.bind("servoMove",
15 }
16
17 void loop() {
18     // On M7, let's print
19     // Buffer it, otherwi
20     String buffer = "";
21     while (RPC.available(
22         buffer += (char)RPC
23     }
24     if (buffer.length() >
25         Serial.print(buffer
26     }
27 }
28
```

RTC RPC

The Real-time Clock (RTC) can be accessed from the M4 core, and using RPC, we can read the values.

In this example, we set the RTC to a specific date (can be adjusted in the example), send it to the M7 via RPC, and finally prints it out in the Serial Monitor using the M7 core.

M4 sketch:

```
1  /**
2  * Initial author: Henat
3  * modified 03 December :
4  * by Hannes Siebeneiche
5  */
6
7  #include "mbed.h"
8  #include <mbed_mktime.h>
9  #include "RPC.h"
10
11  constexpr unsigned long
12  unsigned long printNow{
13
14  void setup() {
15      if (RPC.begin()) {
16          RPC.println("M4: Re
17          RTCset(); //sets th
18      }
19  }
20
21  void loop() {
22      if (millis() > printN
23          RPC.print("M4 System
24          RPC.println(getLoca
25          printNow = millis()
26      }
27  }
28
```

The M7 sketch is found below, and uses RPC to print out the incoming data from the M4.

M7 sketch:

```
1  /**
2  * Initial author: Henatu
3  * modified 03 December 20
4  * by Hannes Siebeneicher
5  */
6
7  #include "mbed.h"
8  #include "RPC.h"
9
10 void setup() {
11     RPC.begin();
12     Serial.begin(9600);
13     while (!Serial) {
14         ; // Wait for Serial
15     }
16     Serial.println("M7: Ser
17 }
18
19 void loop() {
20     if (RPC.available()) {
21         char incomingByte = R
22         Serial.write(incoming
23     }
24 }
```

MicroPython RPC LED

This example demonstrates how to use MicroPython (running on the M7 core) to remotely control an LED from the M4 core.

M4 sketch:

```
1 #include "RPC.h"
2
3 void led(bool on) {
4     if (on) {
5         digitalWrite(LEDG, LOW);
6     } else {
7         digitalWrite(LEDG, HIGH);
8     }
9 }
10
11 void setup() {
12     pinMode(LEDG, OUTPUT);
13     RPC.bind("led", led);
14     RPC.begin();
15 }
16
17 void loop() {
18
19 }
```

M7 Python script:

```
1 import time
2 import msgpackrpc
3
4 # Create an RPC object
5 rpc = msgpackrpc.MsgPackRPC()
6
7 # Start the remote process
8 rpc.start(firmware=0x0818)
9
10 # Toggle the LED every 500ms
11 while True:
12     rpc.call("led", True)
13     time.sleep_ms(500)
14     rpc.call("led", False)
15     time.sleep_ms(500)
```

RPC Library API

The `RPC` library is based on the `rpclib` C++ library which provides a client and server implementation. In addition, it provides a method for communication between the M4 and

This library is included in the GIGA Board Package, so it is automatically installed with the Board Package. To use this library, you need to include

`RPC.h` :

```
1 #include <RPC.h>
```

RPC.begin()

Initializes the library. This function also boots the M4 core.

Syntax

```
1 RPC.begin()
```

Returns

1 on success.

0 on failure.

RPC.bind()

Used on the server side to bind a name to a function, and makes it possible for remotely calling it from another system.

Syntax

```
1 RPC.bind("this_function",
```


`"name_of_func"` - name given for the function to be called from the client side.

`name_of_func` - name of the function on the server side.

Returns

None.

RPC.call()

Used on the client side to call a function with optional parameters.

```
1 RPC.call("this_function",
```

Parameters

`"name_of_func"` - the name of the function declared on the server side.

`args` - arguments to be passed to the function.

Returns

Result of the function if arguments are passed.

RPC Serial API

The RPC Serial methods are also included in the `RPC` library, and uses methods from the `Stream` base class, and is similar to the `Serial` class.

As the `Serial` class is only available on the M7 core, the M4 core uses `RPC` library to print data, where the

RPC.println()

Prints data to a serial port. This is used on the M4 core to send data to the M7.

Syntax

```
1 RPC.println(val);
```

Parameters

The value to print. Can be any data type, but not multiple (e.g. string + integer in the same call).

Returns

Number of bytes used. E.g. printing ("hello") returns 7. As hello (5) + new line (2) = 7.

RPC.available()

Get the number of available bytes to read from the M4.

Syntax

```
1 RPC.available();
```

Parameters

None.

The number of bytes available to read.

`-1` if there is none.

RPC.read()

Reads the first available byte from the M4.

Syntax

```
1 RPC.read();
```

Parameters

None.

Returns

The first available byte from the M4.

`-1` if there is none.

RPC.cpu_id()

Returns the current CPU ID, which can be one of two values:

`CM7_CPUID` for the Cortex-M7 core, or `CM4_CPUID` for the Cortex-M4 core.

Syntax

```
1 RPC.cpu_id();
```

Parameters

Returns

`CM7_CPUID` for the Cortex-M7 core.

`CM4_CPUID` for the Cortex-M4 core.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

